

Executive Summary

The Serial Peripheral Interface (SPI) is a ubiquitous synchronous serial communication standard originally developed by Motorola and now used across embedded and SoC designs. It provides a simple 4-wire, full-duplex link (with 8-bit or variable-length frames) between one master and one or more slave devices. Unlike I²C or UART, SPI can achieve higher speeds (tens of MHz) and true simultaneous send/receive, at the cost of extra chip-select lines and no built-in arbitration. Modern SPI variants include dual, quad, and octal modes (using multiple data pins) for higher throughput, especially in flash memories. This reference compiles Motorola/NXP SPI guides, ARM SSP docs, IP manuals, flash datasheets (Winbond, Micron), and verification literature to present end-to-end SPI

concepts for verification engineers. It covers the protocol history, bus architecture, timing details, modes 0–3, transaction examples (including dummy-cycle protocols), advanced forms (3-wire, dual/quad/octal, daisy-chain, multi-master), internal controller RTL and FSM, common design bugs, and verification strategies (UVM agents, SVA, coverage) — all with examples, diagrams, and code snippets. This single document is intended as a one-stop SPI reference.

1. Introduction, History, and Protocol Comparisons

What is SPI? The SPI bus is a master-slave, synchronous serial interface defined in the 1980s by Motorola (now NXP). It consists of a

clock (SCLK), master-output/slave-input (MOSI), master-input/slave-output (MISO), and a slave-select (SS_n) line for each slave. The master drives the clock and one SS line low to select a slave; simultaneously the master outputs bits on MOSI while the slave sends bits on MISO. After each clock edge, both sides shift their shift registers – effectively exchanging one bit per clock in both directions (“full-duplex”). In practice, 8-bit transfers are common, but many SPI systems allow arbitrary or configurable word lengths up to 128 bits or more.

Protocol Variants: The original “Motorola SPI” mode (now often called SPI Mode 0–3) is widely used; other variants include Texas Instruments’ SSI format (bit phase/polarity renamed SPO/SPH), and Intel’s enhanced SPI. The SPI bus is often preferred over I²C for higher speed and simplicity (no multi-master arbitration, push-pull

drivers), and over UART for deterministic multi-slave daisy-chaining. However, SPI uses more I/O pins and has no acknowledgement bit, so care must be taken with slave-select and multi-master scenarios. Table 1 compares SPI to related interfaces:

Featur e/Bus	SPI (Motor ola)	I ² C	UART	[e.g.] 3- Wire SPI
Wires	4 (SCLK, MOSI, MISO, SS_n)	2 (SDA, SCL) (open- drain)	2 (TX, RX)	3 (CLK, SDIO, SS)
Mode	Master -slave, multi-	Multi- master, multi-	Point- to- point,	Master -slave half-

	slave (bus) or daisy- chain	slave (bus)	no select	duplex
Speed	Higher (tens of MHz typical)	Lower (≈100– 400 kHz typical)	Varies, high- bit- rate (baud)	Similar to SPI
Duplex	Full- duplex (simult aneous)	Half- duplex (share d SDA)	Full- duplex (TX/ RX separa te)	Half- duplex
Topolo gy	Multidr op	Multidr op	Point- to-	Like SPI

	(each SS_n) or daisy-chain	with arbitration (ACK/NACK)	point	with shared I/O
Complexity	Simple (no arbitration)	Complex (bus arbitration)	Simple (only 2 devices)	More complex logic for bit control
Clock Mode	Configurable CPOL/CPHA (modes 0–3)	Data valid on SCL edges (phase fixed)	Asynchronous, start/stop bits	Similar to standard SPI
Exempl	Sensor	EEPROM	UART:	Startup

e Uses	s, SPI Flash, ADCs/ DACs, display s	M, sensor s, PMIC	RS232, logging	EEPRO Ms, simple sensor s
--------	--	----------------------------	-----------------------	---------------------------------------

Table 1: Comparison of SPI with other serial interfaces and 3-wire SPI. SPI's full-duplex and flexible clocking suit many embedded links.

History and Standards: Although widely used, SPI has never been formally standardized by an international body; instead its “official” descriptions come from MCU vendor docs and app notes. Key references include Motorola’s AN991 (1994), NXP’s SPI Block Guide, ARM’s PL022 SSP (Synopsys designWare)

documentation, and numerous microcontroller reference manuals (ST, TI, NXP, etc.). These are the primary sources for detailed behavior and quirks. (Notably, evidence suggests SPI did not literally first appear with the 68000 in 1979; it emerged later on 8-bit MCUs.) The SPI protocol has evolved via extensions: 3-wire variants where MOSI and MISO are time-multiplexed, dual/quad/octal SPI for multi-bit I/O, daisy-chain configurations (all slaves in series), and double-data-rate (DDR) modes. This document consolidates official SPI specs (Motorola/NXP, ARM, TI, Synopsys) and flash memory protocols (Winbond, Micron) into one authoritative guide.

2. SPI Architecture and Signals

2.1 Basic Master–Slave Interface

An SPI bus involves exactly one active master and one or more slaves. The master generates the serial clock (SCLK) and drives slave-select lines to activate a particular slave. Each slave has a dedicated SS_n (active-low chip-select) input. Figure 1 illustrates a simple SPI link:

flowchart LR
Master((Master SPI)) -- MOSI --> Slave((Slave SPI))
Slave -- MISO --> Master
Master -- SCLK --> Slave
Master -- SS_n --> Slave

Figure 1: SPI master–slave connections. Master drives SCLK and MOSI, reads MISO; asserts SS_n low to select the slave.

All SPI devices share the SCLK and MOSI lines; only the selected slave drives the MISO line, while others must tristate MISO to avoid bus contention. As AN991 puts it: “Every SPI system

consists of one master and one or more slaves". The four basic signals are MOSI (master-out/slave-in), MISO (master-in/slave-out), SCLK, and SS_n. For clarity:

MOSI (Master Out, Slave In) : Data output from the master (or input to the master in simplex modes). Master writes to MOSI; slaves read from MOSI.

MISO (Master In, Slave Out) : Data output from the selected slave (or input to the slave). Slave writes to MISO; master reads from MISO.

SCLK: Serial clock line, driven by the master. All data bits (on MOSI/MISO) are sampled or shifted on SCLK edges as configured.

SS_n (Slave Select) : Active-low select line, driven by the master. Only the slave with SS_n = LOW is active to drive/receive data.

All other slaves ignore SCLK/MOSI and tristate their MISO.

Internally, each SPI port uses a shift register (often double-buffered via transmit/receive FIFOs) to handle serial-to-parallel conversion. On each clock edge, the master shifts out one bit on MOSI while the slave shifts out one bit on MISO, then both sample the received bit. As Wikipedia notes, *“during each SPI clock cycle, full-duplex transmission of a single bit occurs. The master sends a bit on MOSI while the slave sends a bit on MISO, and then each reads their corresponding incoming bit”*. Thus in hardware, both sides have 8-bit (or wider) shift registers: after 8 SCLK pulses, an 8-bit word has been exchanged between master and slave, even if one side was sending dummy bits. The shift registers essentially form an *inter-chip circular*

buffer. Figure 2 illustrates a common SPI controller block diagram.

```
graph TD
    CPU[CPU/DMA] -->|Writes| SPI_CTRL[SPI_CTRL]
    SPI_CTRL -->|Config| CLK_DIV[CLK_DIV]
    SPI_CTRL -->|Config| CPOL_PHA[CPOL_PHA]
    SPI_CTRL --> TX_FIFO[TX_FIFO]
    SPI_CTRL --> RX_FIFO[RX_FIFO]
    SPI_CTRL -->|IRQ| IRQ[IRQ]
    IRQ -->|Interrupts| IC[Interrupt Controller]
    CLK_DIV --> SCLK[SCLK Line]
    TX_FIFO --> SR[Shift Register]
    SR -->|MOSI| Slave[Slave]
    Slave -->|MISO| SR
    SR --> RX_FIFO
```

Figure 2: Simplified SPI master controller architecture. The master CPU configures SPI_CTRL, which sets CPOL/CPHA and clock divider, loads data into TX FIFO, and reads RX FIFO. The shift register handles the serial I/O on MOSI/MISO.

Data Framing and Order: SPI data is typically

transmitted MSB-first. The application note states explicitly that MOSI and MISO “are used to transfer data serially... Data is transferred most significant bit first”. (Some controllers allow LSB-first, but MSB-first is the default Motorola convention.) Data words can be 8 bits, 16 bits, or larger; common sizes are [8, 16](#), or 32 bits, but some IPs support up to 128 bits or arbitrary lengths. After a complete word is shifted, the shift register may reload for another transfer, or an interrupt/flag signals completion.

Clock and Select Behavior: A key design is that SCLK idles at a defined level (logic 0 or 1) when no transfer is ongoing, and transitions during active transfer. When the master asserts SS_n LOW, it signals the start of a transfer. Data bits are then clocked out on one edge of SCLK and sampled on the other (see Section 4). When the master deasserts SS_n HIGH after the transfer,

the slave immediately stops driving MISO and returns to idle state. In a one-slave system, SS_n may be tied low; with multiple slaves, each slave needs its own SS_n line (multidrop mode) or the SPI controller can be in daisy-chain mode (only one shared SS_n).

Example from Motorola AN991: The original Motorola SPI app note (AN991) provides a concrete example. It notes: *“Four basic signals, MOSI, MISO, SCK, and SS, are needed for an SPI”*. It describes that MOSI is an output on the master and input on the slave (data flow master → slave), and MISO is input on master, output on slave (slave → master). It also details that exactly one bit is transferred per clock pulse (“one byte in eight clock cycles”). This reaffirms the core behavior: *one bit per clock, MSB first, full-duplex exchange*.

2.2 Master/Slave Roles and Multi-Slave

The master is the SPI controller that generates SCLK and initiates transfers. It writes data to its TX buffer, starts the clock, and reads received data from RX buffer. The slave simply shifts data in/out synchronized to SCLK whenever its SS_n is asserted. If multiple slaves are present, *only the selected slave drives MISO*; all others must tristate MISO. If a slave cannot tristate its output, external tri-state buffers are needed to avoid bus conflicts.

When a master raises SS_n (goes HIGH), it deselects the slave and typically ends the transfer. The SPI block in the microcontroller often has a “SS output” feature: *“the SS output feature automatically drives the SS pin low*

during transmission... and drives it high during idle". Conversely, if the master's SS_n pin is configured as an input (MODE fault feature), then any unexpected low on that pin (from another master) generates a *mode fault* error. This is used in multi-master systems to detect bus contention: *"if the SS input becomes low... this indicates a mode fault error where another master tries to drive MOSI".*

For multi-master support (rare in SPI), both masters must monitor SS pins and tri-state lines, or implement arbitration protocols. Typically SPI is single-master; if multiple controllers need shared devices, bus arbitration (e.g. round-robin or chip-select arbitration) must be implemented at a higher level.

For multi-slave topologies, there are two common schemes:

Multidrop (parallel) – Each slave has its own SS_n line. The master activates exactly one SS_n per transfer. MISO/MOSI/SCLK lines are shared, and only the selected slave responds. This is the most common configuration.

Daisy-chain (serial) – All slaves are connected in series for the data path: the master's MOSI feeds SLAVE1's MOSI, and SLAVE1's MISO feeds SLAVE2's MOSI, etc., with the last slave's MISO returning to the master. Only one SS_n line is needed for the chain. In this mode, data shifts through all devices sequentially. Daisy-chain can save SS pins but is more complex and less common. The Wikipedia SPI page notes that daisy-chain mode “only requires one SS signal”, while multi-drop needs one per device.

In either case, proper SS_n timing is crucial: all slaves must see SS_n HIGH except the intended one. Unselected slaves must *ignore* any clock or data (they effectively drop SCLK and MOSI), and release MISO. In practice, designers often ensure any shared line (like MISO) is pulled to a known state (via pull-ups or tristate logic) when idle.

2.3 Full-Duplex Data Transfer (Shift Registers)

SPI's hallmark is *full-duplex transfer*, meaning one bit is sent by master and one bit sent by slave simultaneously on each clock. Internally, each side uses a shift register (often tied to a TX and RX FIFO) to effect this. At the start of a transfer, the master loads a byte (or word) into

its TX shift register and asserts SS_n low. As SCLK pulses, the register shifts out bits MSB-first onto MOSI, one per clock. Simultaneously, the slave's shift register (preloaded with a response byte or dummy data) shifts bits onto MISO. After each bit, both master and slave latch the incoming bit into their RX register.

In effect, *each clock pulse yields one transmitted bit and one received bit*. After 8 clocks, the master's transmitted 8-bit word reaches the slave, while the slave's word reaches the master. If longer data is to follow, the shift registers reload (or FIFOs continue the stream) and SCLK keeps running. AN991 explains: *“one bit of data is transferred every clock pulse, and one byte can be exchanged in eight clock cycles”*.

Crucially, even in a write-only transfer (master

sending data to slave), the slave must still shift something back (often dummy zeros) on MISO. Likewise, in a read-only scenario, the master shifts dummy bits (e.g. 0x00) on MOSI to generate clocks for the slave to send real data. In all cases, the exchange is bidirectional at the hardware level. This can be summarized:

On the first SCLK edge after SS_n assert, both devices present the first data bit on their outputs.

On subsequent edges, next bits appear.

Typically one edge is used to *shift* data out (drive the line), and the opposite edge is used to *sample* the incoming line (read into the register).

After 8 edges, a full byte has been sent and received. The SPI controller then usually flags “transfer complete” (and can optionally

trigger an interrupt or reload the registers for the next byte).

According to Wikipedia, *“the master sends a bit on the MOSI line while the slave sends a bit on the MISO line, and then each reads their corresponding incoming bit”*. The same reference notes that if the data size is less than 8 bits, the same principle applies (only fewer shifts).

3. Electrical and Timing Considerations

3.1 Signal Integrity and Timing Parameters

As a high-speed interface, SPI timing must be carefully observed. Key timing parameters (from flash datasheets or controller specs) include:

Setup time (t_{SU}): The time that MOSI (or address lines) must be stable before the SCLK sampling edge.

Hold time (t_H): The time MOSI must remain stable after SCLK's sampling edge.

Clock-to-out (t_{CO}): Delay from SCLK edge to valid data on MISO.

Propagation delays: Both MOSI and MISO lines may have bus capacitance; signals should be driven fast enough to meet required edges.

SCLK phase/edge jitter: The data sheet of each device will specify maximum clock rise/fall times (t_R , t_F) and maximum frequency (f_{MAX}). For example, Winbond flash

specifies $t_{\text{CLH}} = 4 \text{ ns}$ (clock low-to-high) for most commands and requires MOSI stable $t_{\text{SU}} = 8 \text{ ns}$ before sampling.

For instance, the Winbond W25Q80AC flash datasheet defines (for standard Read 0x03): setup/hold times of a few ns and requires the chip select be asserted prior to data, etc.. When designing an SPI controller, ensure that the chosen clock divisor yields SCLK edges that satisfy both master-side flip-flop times and the target device's AC timings.

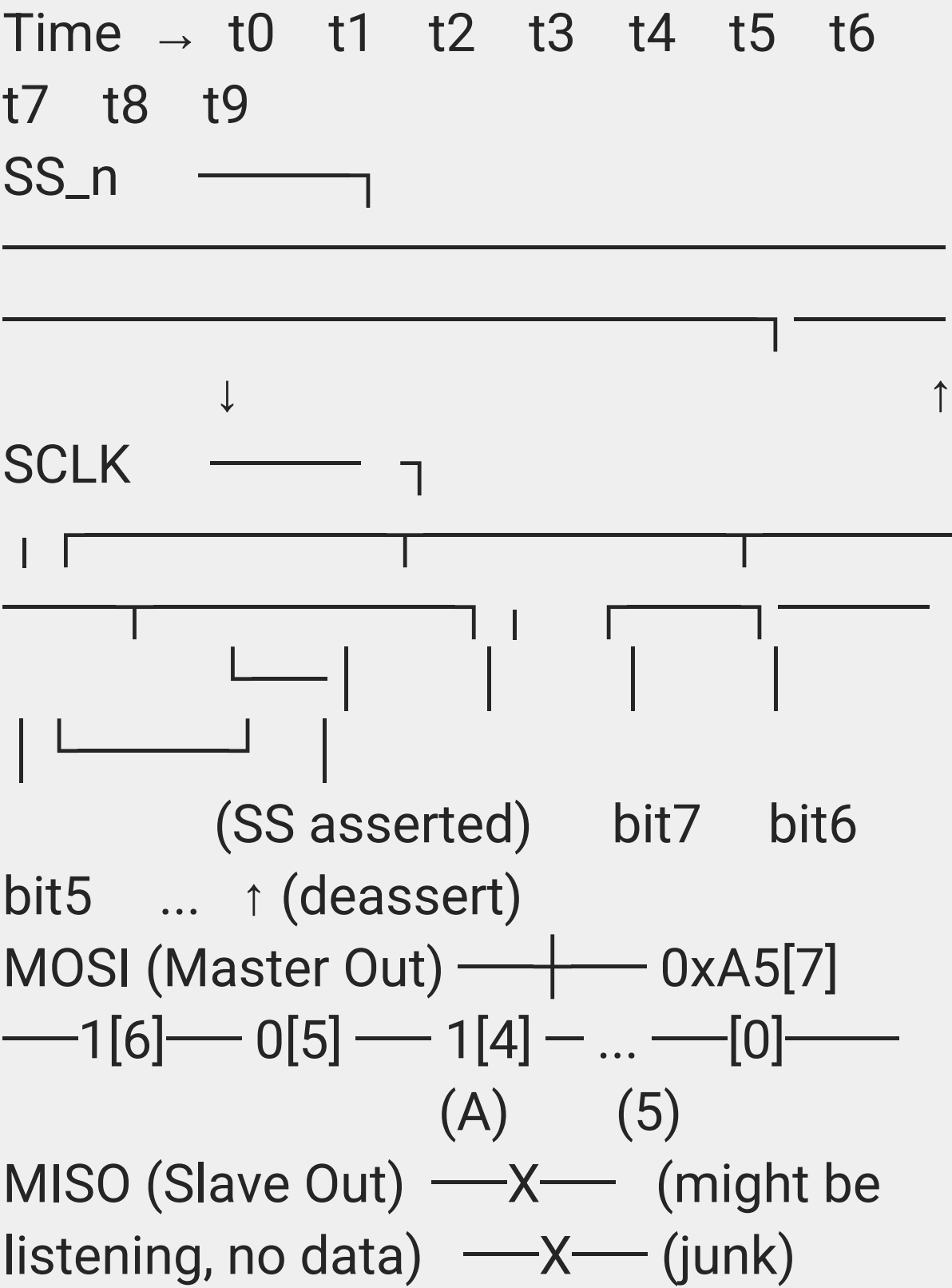
Metastability and Synchronization: Often SCLK and SS_n are asynchronous with the system/CPU clock. To avoid metastability in the SPI controller, SCLK crossings should be synchronized before sampling. Many SPI controller designs use multi-flop synchronizers

on SS_n and edges of SCLK (or simply drive the controller with the same clock as SCLK in FPGA/ASIC implementations). Likewise, if MISO data goes directly to logic in another clock domain, synchronize it. Failure to synchronize can cause unpredictable behavior. While not specific to SPI (this is general CDC), engineers must treat SPI lines as asynchronous inputs and meet their minimum setup/hold requirements in the FPGA/ASIC fabric. A poorly designed SPI interface can suffer data jitter or metastability on sampling edges.

3.2 Timing Diagrams (Setup/Hold)

Below is an example ASCII timing diagram for a simple Mode 0 transfer (CPOL=0, CPHA=0) of one byte (0xA5) from master to slave. Here SCLK idles low; data is shifted on falling edges

and sampled on rising edges:



(Simple Mode 0 example: Master pulls SS low, then on each falling SCLK (▼) the master shifts out the next MOSI bit. Slave samples at each rising edge (▲) and shifts its MISO (not shown). MOSI bits 1-0-1-0-0-1-0-1 correspond to 0xA5.)

Key points: the setup time (MOSI stable before SCLK rising) and hold time (after SCLK rising) must meet the slave's requirements. In Mode 0, data is valid around the rising edge. The diagram shows SS asserted before t₁ (ensuring SS low before first clock) and SS deasserted after the last bit and clock.

Real controllers provide registers to choose CPOL/CPHA and divisors. Designers should consult device timing tables. For instance, Winbond's Fast Read (0Bh) instruction requires 8 dummy cycles after the address (Figure 10), ensuring the flash has time to prepare data

internally. The diagram below (excerpt from [28]) highlights this dummy-clock concept:



Winbond Fast Read (0Bh) Sequence:

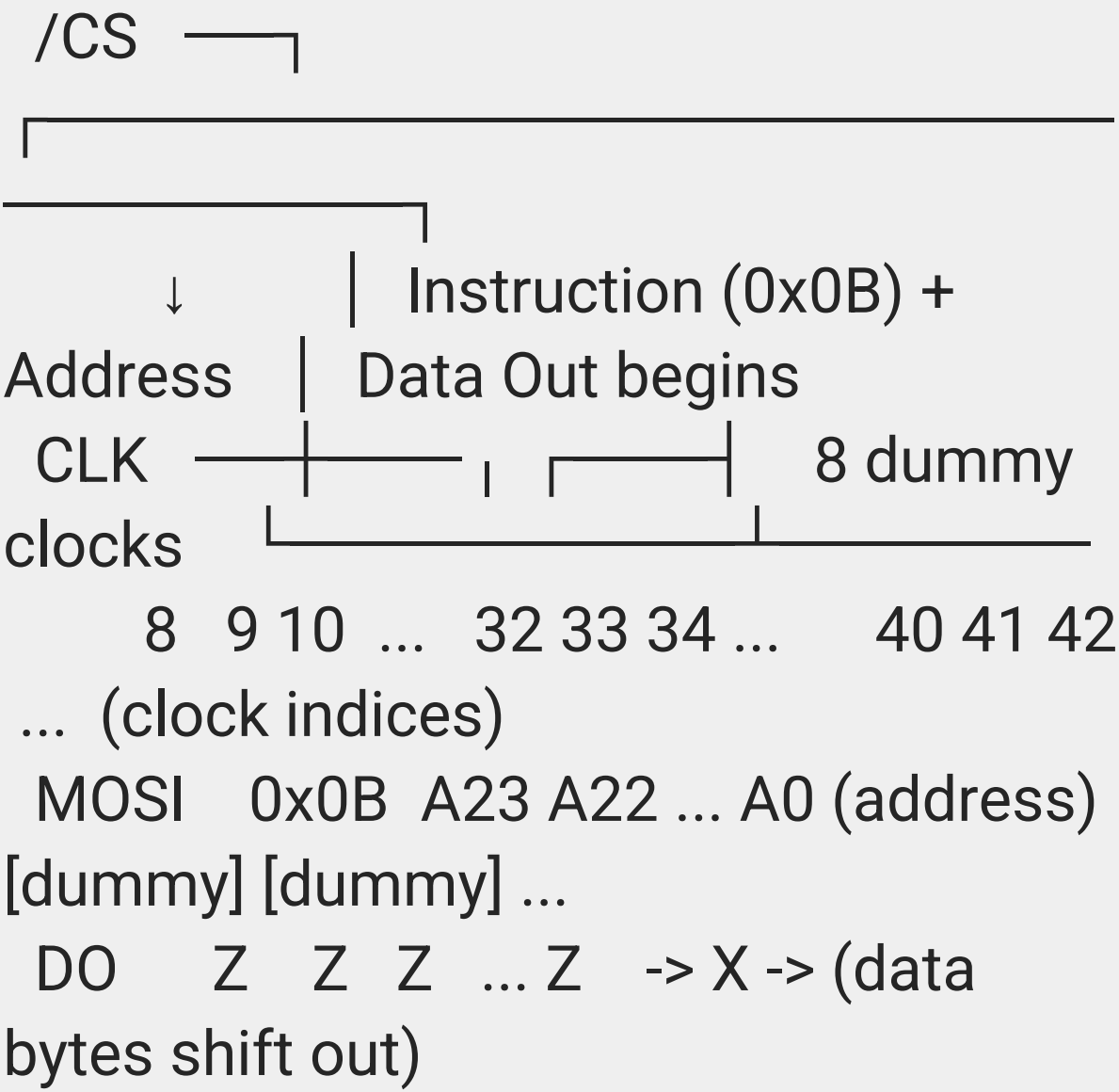


Figure 3: Winbond Fast Read (0x0B) with 8 dummy clocks after the address. (Z=High-Z

during instruction, X=don't care; data starts at the 9th rising clock after address.)

In summary, designers must align SS, SCLK, MOSI, and MISO timing to device specs. Use timing diagrams (like above) and the memory controller's tables (e.g. t_{setup} , t_{hold} , t_{access}) for correct operation.

4. Clock Polarity and [Phase \(SPI Modes 0–3\)](#)

SPI communication relies on two parameters: clock polarity (CPOL) and clock phase (CPHA) . These are often called SPO/SPH or CKP/CHE in various manuals, but the Motorola convention labels them CPOL and CPHA. They define when data is sampled relative to the clock edges.

There are 4 combinations (modes 0–3):

Mode 0 (CPOL=0, CPHA=0): Clock idles low. Data is sampled on the first (rising) edge after SS_n goes low. On the immediate rising edge, the first data bit is captured. Master shifts out each bit on the falling edge. This is the classic mode.

Mode 1 (CPOL=0, CPHA=1): Clock idles low. Data is sampled on the second (falling) edge. The master begins shifting data on the first rising edge, but the slave doesn't sample it until the following falling edge.

Mode 2 (CPOL=1, CPHA=0): Clock idles high. This is essentially like Mode 0 but inverted clock. Data is sampled on the first falling edge after SS_n, with shifting on rising edges.

Mode 3 (CPOL=1, CPHA=1): Clock idles high.

Sample on second edge (rising) after SS_n low. Shifting occurs on falling edges.

Table 2 summarizes this:

SPI Mode	CPOL	CPHA	Clock Idle Level	First Shift Edge	D
0	0	0	Low	First falling (▼)	F
1	0	1	Low	First rising (▲)	S
2	1	0	High	First	F

				rising (▲)	fa (
3	1	1	High	First falling (▼)	S d ri (

Table 2: SPI mode definitions. CPOL sets idle SCLK; CPHA selects whether data is sampled on the first edge (0) or second edge (1) after SS_n asserts. (Based on [48] and [62] timings.)

In [Mode 0 \(CPOL=0, CPHA=0\)](#), the master should output the first data bit *immediately* when SS_n goes low, as the TI app note explains: “When SPH=0, data is captured on the first clock edge.” For Mode 1 (CPHA=1), the data is output on the first edge but only captured on the *next*

edge. Figures in [48] (Texas Instruments) show these timing differences. The Wikipedia SPI timing diagram (Figure 12 in [62]) also illustrates CPOL/CPHA effects: mode 0 data lines are active on the first edge (blue vs red in [62†L356-L365]), whereas mode 3 data aligns on the second edge.

Mathematical View: A convenient way to remember modes is as a 2-bit value: $\text{mode} = (\text{CPOL} \ll 1) \mid \text{CPHA}$. That is, Mode 0 = (0,0), Mode 1 = (0,1), Mode 2 = (1,0), Mode 3 = (1,1). Engineers often say “SPI mode 0” to mean CPOL=0/CPHA=0, etc. Changing modes effectively inserts a half-cycle delay between when SS_n goes active and when data is sampled.

Comparison and Tables: Table 2 above captures the main difference in terms of active edges.

Many sources (including vendor docs) label these as Motorola modes 0–3. An alternate notation is (CPOL,CPHA); for example, (0,1) is mode 1. Practically, the slave device's datasheet will specify which mode to use. If a master and slave misconfigure mode, data will be sampled on the wrong edge and transfers will fail. SPI masters often allow changing CPOL/CPHA in hardware or software between transactions.

5. SPI Transactions: Sequences and Examples

A *transaction* is a complete exchange from the master's perspective. In SPI, this typically means the master asserts SS_n low, transfers some clocks with data, then deasserts SS_n high. Common transaction types include simple

register reads/writes or memory accesses. Each device defines its own protocol above the raw bits. We discuss standard single-byte and multi-byte SPI exchanges, using examples from an SPI flash (Winbond W25Q80) and a generic controller.

5.1 Single-Byte Read/Write Examples (SPI Flash)

Consider a serial flash memory using “Motorola” SPI commands. The simplest read command is typically opcode 0x03 (Read Data). The Winbond W25Q80 flash doc describes: *“The Read Data instruction is initiated by driving /CS low and then shifting the instruction code ‘03h’ followed by a 24-bit address... The data byte of the addressed memory location will be shifted out on the DO pin at the falling edge of CLK with*

MSB first". In other words, the master sequence is:

1. Drive SS_n LOW.
2. On MOSI, send 0x03.
3. Continue 24 clock pulses, sending the 24-bit address (MSB first) on MOSI.
4. For each subsequent clock, read a data bit on MISO (the slave will output memory data).
5. Repeat clocks to read more bytes; the address auto-increments after each byte.
6. Deassert SS_n HIGH to end the command.

Clock-by-Clock Example: Suppose we want to read 3 bytes at address 0x001234. The sequence (Mode 0) is:

MOSI: 0x03 = 00000011 (MSB first)

MOSI: Address 0x001234 = 00000000

00010010 00110100 (24 bits)

Then 3 bytes of data come back on MISO.
 Table 3 sketches the bus for the first few cycles:

Cycle (CLK edges)	MOSI Out (master → slave)	MISO In (slave → master)	Operation
1–8	00000011 (0x03 opcode)	– (Z, slave idle)	Send READ opcode
9–16	00000000 0 00010010 0 00110100 0	– (Z)	Send 24- bit address

17–24 (bits 23–16)	<i>First data byte (out)</i>	<i>First data byte (in)</i>	Data[23:16] → MISO (MSB first)
25–32 (bits 15–8)	(don't care or next addr)	<i>continues data</i>	Data[15:8] → MISO
33–40 (bits 7–0)	–	<i>continues data</i>	Data[7:0] → MISO (LSB first)
(Repeat clocks for more bytes)	N/A	<i>Byte 2 out on MISO</i>	Next address (0x001235) read

(Repeat clocks)	N/A	<i>Byte 3 on MISO</i>	Next address (0x00123 6) read
SS_n High			End transacti on (CS rises)

Table 3: Clock-by-clock example of an SPI flash Read (0x03) command, reading 3 bytes starting at 0x001234. Data is output on MISO (yellow) on falling edges as MSB-first. MOSI is driven by the master (blue), first sending the opcode and address.

Winbond’s document confirms that continuous reading is allowed: “the address is automatically incremented to the next higher address after each byte... allowing a continuous stream of

data. The instruction is completed by driving /CS high". Thus a single CS_low → 0x03 → address → N clocks → CS_high sequence can read arbitrarily many bytes (up to end of memory, or until CS rises).

For writing (page program), the flow is similar but with different opcodes. For example, Winbond's 0x02 Page Program command is initiated similarly: *"driving the /CS pin low then shifting the instruction code '02h' followed by a 24-bit address and at least one data byte... The sequence is shown in Figure 16"*. In practice, the master would first send a Write Enable (0x06) command, then /CS=0 → 0x02 → address → data bytes → /CS=1 to program up to 256 bytes (one page). Figure 16 in [74] (not shown) depicts this: clocks shift out address then data, and upon CS high the flash begins internal programming. The slave (flash) ignores writes if

not already enabled, which the datasheet warns about.

5.2 Fast Read and Dummy Cycles

Many SPI flash chips support a Fast Read (0x0B) or similar, allowing higher clock rates. The difference is insertion of *dummy cycles* after the address. From Winbond: *“Fast Read is similar to Read Data except that it can operate at the highest frequency... by adding eight ‘dummy’ clocks after the 24-bit address... During the dummy clocks the data value on DO is “don’t care”.*

Figure 4 (from [28]) shows that after sending 0x0B and address, the master issues 8 more clocks (MOSI high-impedance or zeros) before the slave outputs the first data byte. This lets the

internal memory array time to fetch the first word. After the dummy cycles, the flash drives data out on MISO each clock.

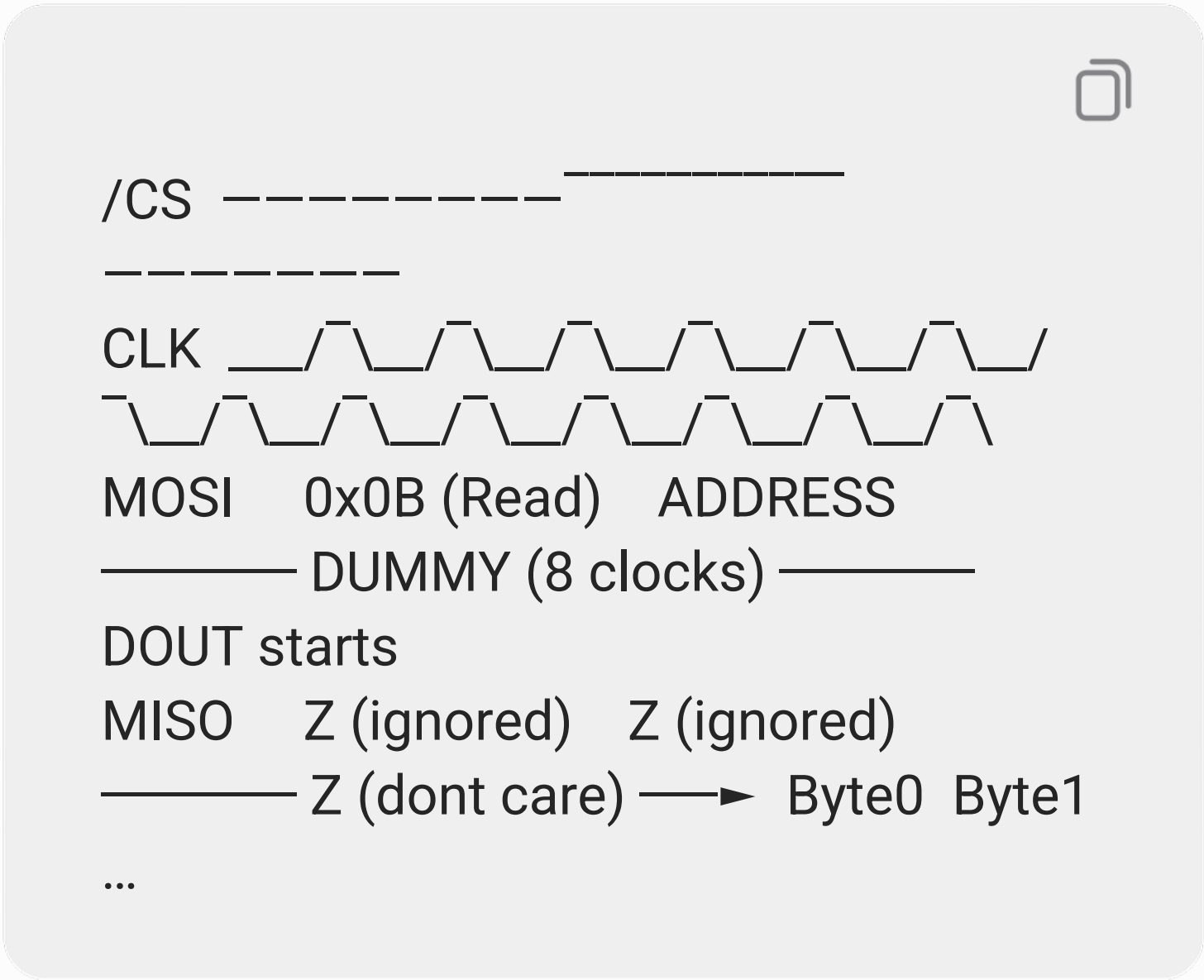


Figure 4: Fast Read (0x0B) sequence on flash: after opcode+address, master clocks 8 dummy cycles before flash outputs data.

Similarly, dual- and quad-I/O read commands output data on multiple pins. For example, *Fast Read Dual Output (0x3B)* outputs data on two lines (IO0 and IO1), doubling the throughput. *Fast Read Quad Output (0x6B)* uses four pins (IO0–IO3) and requires enabling the flash’s QE bit. Both dual and quad reads also include dummy cycles after the address for high-speed operation. These commands are essential in modern high-speed SPI Flash integration.

5.3 Multi-Byte and Burst Transfers

Beyond single reads/writes, SPI supports *burst* operations. In MCU controllers, one can often configure the SPI peripheral in “continuous” or “multiple” mode, where as long as SS_n stays asserted and clocks are present, bytes keep shifting. For example, a microcontroller might

allow sending or receiving a stream of N bytes by filling a FIFO or enabling DMA. The flash examples above show continuous bursts of data after the initial command/address.

In practice, a SPI master will often:

- Set up SS_n low, issue the command and address as above.

- Toggle SCLK for the needed number of bits (possibly thousands if reading many bytes).

- Optionally leave SS_n low between bytes in a continuous transfer, or deassert/resassert for separate transfers. If CS goes high too soon, the slave aborts the burst (and often requires restarting).

- Use FIFO or DMA to handle large bursts without CPU intervention (section 7).

For reads from flash or memory, engineers must

be mindful of bus turnaround (e.g. don't drive MOSI after address if not needed) and align reads on clock edges. Many SPI controllers let you program how many bits/bytes to transfer and will automatically generate the appropriate clocks and reassert final SS_n.

5.4 Miscellaneous Transfer Types

Register Reads/Writes: Many devices (sensors, ADCs, DACs) use custom opcodes for reading/writing registers. The pattern is the same: assert CS, send op code (often with R/W bit), send register address, then read or write data bytes. Always consult the device's SPI command set.

Burst Read/Write: Some SPI devices support multi-byte bursts where after the first address, the device auto-increments an

internal pointer. This is common in memory or streaming sensors.

Continuous Streaming: As noted, keeping SS_n low allows the master to stream data (as long as the slave will auto-respond). For example, rotating a shift register through a chain of SPI devices can be done by continuously clocking.

Command-Only Transactions: Some use cases involve toggling lines without sending useful data (e.g. mode detection, or bit-banging SPI in software). These are discouraged unless necessary.

In all cases, timing of CS relative to SCLK is crucial. Most slaves require CS to be stable (high) for some time before asserting low, and require a minimum SS pulse width to frame a transfer. These appear in datasheet as tCSS (CS

setup) and tCSH (CS hold) parameters.

6. Advanced SPI Variants and Flash Protocols

SPI has many variants beyond the classic 4-wire, single-bit mode. Verification engineers should be aware of common extensions and custom interfaces derived from SPI.

6.1 Three-Wire (Half-Duplex) SPI

Some simple or legacy devices use a 3-wire SPI, where MOSI and MISO are time-multiplexed on a single bidirectional data line (often called SIO or SDIO). In 3-wire mode:

The master drives the data line to write to

the slave.

The slave drives the same line to send data back.

This saves one pin (only MOSI/MISO line), but requires half-duplex logic (the master and slave take turns driving the line). During a transaction, the master may switch its MOSI pin to input mode when reading back. The Wikipedia SPI page notes that in 3-wire mode “the MOSI line becomes SIO0 and carries even bits, while the MISO line becomes SIO1 and carries odd bits” in dual-edge modes (though that quote is for a specific DDR mode). In practice, 3-wire SPI is uncommon in hardware IP; often it is manually bit-banged (especially for trivial boot ROM loads). If supported, one must carefully manage the direction of the single data pin and ensure no bus conflict.

6.2 Dual and Quad SPI (Multi-IO)

For higher data rates, dual-SPI and quad-SPI use multiple I/O pins after the command. There are two main kinds:

Dual/Quad Output (DO/QO): After the address, data is output on 2 or 4 lines simultaneously. E.g. Winbond's 0x3B (Dual Output) and 0x6B (Quad Output) commands. These still send the address on MOSI (single line), then switch to multiple lines for data output. They require the flash to have a "dual/quad enable" flag (e.g. QE bit) and often use dummy cycles. Throughput is 2× or 4× normal.

Dual/Quad I/O: Even more complex, these use multiple lines for both address and data. For example, commands 0xBB (Dual I/O

Read) or 0xEB (Quad I/O Read) use 4 lines for both sending the address and reading data (the instruction opcode is sent on MOSI, then mode switches to I/O lines). They often allow double data rate (DDR) transfer by using both clock edges.

Octal-SPI (OSPI): An emerging extension with 8 data lines, used in new NOR flash (e.g. Micron OCTAL SPI). The command set extends the same idea (8× throughput). OSPI is supported by advanced controllers (e.g. Xilinx Zynq OSPI).

Implementing these requires the controller to tri-state/reserve all I/O lines and manage multi-line shifts. Verification of these modes is more complex (see Sect. 9–11).

6.3 Daisy-Chain and Multi-Master

As noted, daisy-chain SPI connects multiple chips in series. Verification must ensure that data shifted out of one chip becomes input to the next. We saw [62†L337-L340] that this uses a single CS for all. In design, a daisy-chain master must provide exactly as many clocks as the total chain length; each device's output connects to the next device's input. Test sequences must verify that chained data emerges in the correct order.

Multi-Master: Rare but possible. In true multi-master SPI, two masters share MOSI, MISO, and each has a mode-fault detection on SS. If both attempt a transfer, one will see a mode fault (SS asserted on its input). A multiprocessor system

might avoid this by allowing bus arbitration via GPIO. Verifying multi-master would involve scenarios where two masters attempt bus access, and checking that only one succeeds (the other detects mode fault). Real bug example: if one master forgets to disable its SPI before another drives the bus, a mode fault can freeze transfers.

6.4 SPI Flash Command Protocols (Dummy Cycles, Address Phases)

Beyond basic read/write commands, SPI flash chips use complex command sequences. Key points for verification:

Dummy Cycles: Fast reads and dual/quad reads require *dummy clocks* after the address (often 8 or more) before data is valid. UVM scenarios should include cases

with and without dummy cycles (and ensure the VIP correctly inserts or consumes them).

Address Phases: Most SPI memories use 3-byte (24-bit) addresses, some support 4-byte addressing for >16MB memories.

Verify both. The address is sent MSB-first immediately after opcode.

Page/Burst Boundaries: Writes must not cross page boundaries (writes wrap within a 256-byte page). A partial page write might require repeated CS toggling. Flash controllers often require a “write enable” command (0x06) before programming.

Tests should cover: attempting writes without WREN (should be ignored), partial page writes, and wraparound behavior.

Busy/WEL Flags: After programming/erasing, the flash sets a BUSY flag in its status register. A well-designed controller or VIP will poll this bit (via 0x05 read

command) before new commands. Tests should include waiting for BUSY clear or running concurrent instructions (should be ignored if BUSY=1). These are SPI flash specifics beyond pure SPI but often part of SoC SPI verification.

6.5 Comparison of SPI IP Features

Industry IP cores (Synopsys DesignWare, ARM PrimeCell SSP, TI McSPI, etc.) offer various features. Common ones include:

- Multiple SPI modes (0–3).

- Configurable bit order (MSB/LSB first).

- Master and Slave roles.

- Single (one-shot) and continuous transfer modes.

- Multi-slave select handling (GPIO, internal

decode up to 32 slaves).

FIFOs or DMA support for streaming data.

Interrupts for TX-empty, RX-full, transfer complete, mode-fault, overrun.

Clock divider to scale from system clock to SCLK.

For example, AnkaSys' SPI VIP notes "All SPI modes are supported... Support two type agent: SPI Master, SPI Slave... Single/Continuous... Multi Slave up to 32... MSB/LSB selection... Pre-defined covergroups". These are hallmarks of a full-featured SPI solution. Designers should ensure the chosen IP can meet the required data rate (clock divider range) and has the necessary FIFO depth or DMA width for the expected throughput.

7. SPI Controller RTL Architecture

An SPI master peripheral typically consists of the following blocks (see Fig. 2):

Clock Generator: A divider takes the system/core clock and divides it to produce SCLK. For example, many microcontrollers allow dividing by powers of two or any integer (e.g. from $f_{\text{core}}/2$ down to $f_{\text{core}}/256$). The divider value is set by the SPI control register. The generated SCLK is gated by SS_n (i.e. no clock when idle).

Shift Registers and FIFOs: A transmit (TX) FIFO or register buffers data bytes to send. When a transfer begins, data is loaded from TX into a shift register. On each SCLK edge, the shift register outputs the MSB on MOSI

(or drives MOSI pin) and shifts its bits. Concurrently, a receive (RX) shift register latches in the MISO bit and shifts. After 8 shifts, the RX register contains a full byte, which is moved into the RX FIFO or data register. TX and RX buffers smooth the handshake between CPU and serial clock domain.

Finite State Machine (FSM): Controls the SPI phases: IDLE (waiting for SS_n low), START (load shift registers), TRANSFER (clocking bits), LAST (handle final bit), and COMPLETE (flag done and interrupt) states. It also sequences CPOL/CPHA logic. Typical registers include a control reg (mode, enable, master/slave select, CPOL, CPHA) and a status reg (busy, TX empty, RX full, mode fault). An example AN991 diagram (Figure [3-3](#) in [68], not shown here) outlines such FSM states.

Slave Select Logic: For masters, there may be an internal SS output controller (auto-drive SS_n during transfers) or it may rely on software to drive a GPIO. Many designs support multiple chip selects. For slaves, the SS pin feeds a detector that gates the FSM (SS high means drive MISO tri-state and go to idle).

Interrupt and Error Handling: Interrupts for TX underrun (no data while shifting), RX overrun (data lost), transfer complete, and mode fault (in multi-master cases) are common. These flags help the CPU respond to data stream events.

Mermaid Block Diagram (Figure 2) showed a typical SPI master; a slave block would be similar minus the clock divider (slave uses external SCLK) and SS is input. Verification

should ensure these blocks operate correctly:
e.g. TX FIFO should not underflow mid-transfer,
SS_n must be deasserted at transfer end, etc.

In summary, the RTL architecture of an SPI master has a control interface (registers/AXI/APB bus to configure it), a clocking section (divider and edge generator), a data path (shift registers, FIFOs, direction logic), and status/interrupt logic. For DMA, the controller might optionally fetch bytes from memory automatically on TX and write RX bytes to memory, triggering transactions upon FIFO thresholds.

8. Design Challenges and Common Bugs

8.1 Clock Domain Crossings (CDC) and Metastability

Because SPI SCLK is typically asynchronous to the core CPU clock, the SPI core's FSM and logic must synchronize critical signals. Designers often run the shift logic entirely on the SCLK domain, using edge detectors to sample SS_n and MOSI. Alternatively, SS_n can be synchronized into the core clock domain with multi-flop synchronizers. Failure to handle CDC can lead to metastability: e.g. SS_n might glitch or arrive halfway through a core clock cycle. The solution is to *double-flop* SS_n in the core clock domain or ensure all SPI activity (state changes) occurs on SCLK edges only. Verified CDC is critical: assertions should check that SS_n glitches do not cause unintended transfers.

8.2 Glitches and Signal Integrity

On shared lines (SCLK, MOSI, MISO), glitches can occur during power-up or line switching. For example, if MOSI is left floating when no master is active, a residual clock edge could cause a slave to mis-sample. Good practice is to use pull-ups/pull-downs or ensure idle values (e.g. MOSI=0) when idle. Some SPI IPs allow configuring an idle MOSI output. Glitches on SS_n (e.g. due to bouncing switches) can prematurely terminate a transfer. A typical bug is inadvertently asserting SS_n too early, chopping off a transaction.

A concrete bug example: Suppose a master finishes sending 8 bits but deasserts SS_n exactly on the last rising edge instead of after it. The slave might see SS_n go high while it is latching the last bit, causing that bit to be lost or

causing the slave to prematurely reset its state. Verification should include checking that *data sampling occurs before SS_n deassert*, and that the final bit is not corrupted.

8.3 FIFO Underflow/Overflow

If a master's TX FIFO empties before the programmed number of bits has been clocked out, garbage bits (0xFF or undefined) will be sent on MOSI, confusing the slave. Likewise, if the RX FIFO becomes full (slave is sending data faster than the core can read it), data may be lost or cause an overflow flag. Verification must include scenarios where the CPU delays reading RX data or refills TX late, and ensure proper stall flags or back-pressure exist. Code coverage should check TX_empty and RX_full conditions.

8.4 Mode Faults and Slave-Select Issues

As noted, if the master drives its SS_n active low while already in slave mode, or if another master activates the bus unexpectedly, a mode fault can occur. A common design bug: not handling the mode-fault interrupt/flag. This can cause the SPI core to hang or misbehave. The fix is to reset the SPI controller or reinitialize its state machine on a mode fault. Verification scenarios should explicitly toggle SS_n on an active master to simulate contention and check that the core reports the error and recovers.

Another issue: In slave mode, if SS_n is allowed to float or is kept low erroneously (e.g. tied to ground), the slave will never release the bus. The design should detect this and either ignore or

reset.

8.5 CPOL/CPHA Misconfiguration

If the SPI mode bits are incorrectly set, all data will be misaligned. A subtle bug: switching modes in mid-burst without toggling SS_n can confuse devices that do not support mode change on-the-fly. Always reassert SS_n when changing mode. A robust design will include a check (via coverpoints/assertions) that CPOL/CPHA are not altered during an active transfer.

9. Verification Strategy (UVM Testbench Design)

A full verification strategy for an SPI interface typically follows UVM best practices: build a

reusable SPI agent, write directed and random tests, and include self-checking components.

Key elements:

SPI Driver/Monitor (Agent): The testbench includes a *driver* that drives the SPI signals. The driver converts transaction objects (type `spi_transfer` with fields like `cpol`, `cpha`, `data_out`) into pin-level toggles on `clk`, `mosi`, and `ss_n`, and collects `miso` into responses. It handles inserting delays, toggling `SS_n`, and obeying mode settings. The *monitor* samples the actual bus signals (`MOSI`, `MISO`, `SCLK`, `SS_n`) and produces sequences of bits or bytes as transaction items. These feed to the scoreboard and coverage. The ANKASYS VIP description confirms that a proper SPI environment has both master and slave agent types, single/continuous transfer

modes, etc..

Sequence/Solver: A sequencer controls which operations to perform. Examples of sequence items: single-byte read, multi-byte burst read, write-then-read checks, mode-change, dummy-cycle read, etc. Randomized sequences should cover different modes (0–3), bit orders, transfer lengths, and error conditions (e.g. start while busy). The environment may also include a scoreboard that compares the observed MOSI/MISO streams against expected values.

Environment Composition: The UVM env may instantiate both master and slave agents (in loopback mode) to test peer behavior. It should connect interfaces

properly: e.g. link the SPI master's MISO to the SPI slave's MOSI and vice versa. The medium article example [31] shows a UVM environment with driver, monitor, scoreboard, coverage (though we don't cite it here, we follow the same idea).

Clocking and Reset: Provide a stable reference clock for the DUT (and any testbench models). Often the SPI interface is synchronous to a clk (which could be either SCLK or a higher-frequency system clock). Ensure reset cycles bring the SPI core to a known idle state (CS high, FIFOs empty, status clear).

Error Scenarios: Include tests for abnormal conditions: mode fault, FIFO overflow (flood

MOSI without reading MISO), partial writes, illegal instructions, etc. Check that the core sets error flags properly and does not propagate them to system unaddressed.

Reusability: A good SPI UVM agent will have configuration registers (like the AnkaSys VIP description) to enable/disable features (LSB-first, delays, etc). Use `uvm_config_db` or factory overrides to parameterize the SPI agent in different tests (e.g. `cfg_cp_mode = 3`, `cfg_msb_first = 0`, `cfg_delay = N`).

Figure 5 illustrates a typical UVM SPI environment structure: a virtual interface connects the SPI signals to both the DUT and the testbench drivers/monitors.

```
flowchart LR
    subgraph TB
        Master_Agent((SPI Master Agent))
        Slave_Agent((SPI Slave Agent))
        Master_Agent -- MOSI --> Slave_Agent
        Slave_Agent -- MISO --> Master_Agent
        Master_Agent -- SCLK --> Slave_Agent
        Master_Agent -- SS_n --> Slave_Agent
        UserSeq[Sequences] --> Master_Agent
        UserSeq --> Slave_Agent
        Scoreboard((Scoreboard/Reference Model))
        Master_Agent -- TxTransactions --> Scoreboard
        Slave_Agent -- RxTransactions --> Scoreboard
    end
```

*Figure 5: Simplified UVM testbench architecture for SPI. A Master agent generates SPI transfers; a Slave agent (or reference model) replies. Monitors feed transactions to a scoreboard for checking correctness. Configuration, sequences, and coverage collectively ensure thorough verification.**

In summary, the verification plan should cover *all*

operational modes, all data lengths, and error conditions. Leverage constrained-random for variable patterns, directed tests for tricky cases, and checkers/scoreboards to automate correctness checking.

10. Assertion-Based Verification (SVA & Formal)

SystemVerilog Assertions (SVA) and formal methods provide additional confidence. Key assertions might include:

```
SS_n Idle State:assert property @(posedge  
clk) disable iff (!rst_n) (CS_high == 1 ->  
spi_state == IDLE);
```

Ensures that when SS_n is high (no transfer), the SPI controller remains in its IDLE state.

SS_n Reactivation:assert property

```
@(posedge clk) disable iff (!rst_n) (CS_high  
##1 (!CS_low) |-> next(spi_state) ==  
START);
```

If SS_n goes low (selected), the controller must enter the START or TRANSFER state.

Clock-Data Relationship: If in Mode 0, for example, we might assert that MOSI only changes on falling edges of SCLK, and is sampled on rising edges. An SVA snippet:

systemverilog



```
property p_mosi_stable;  
  @(posedge clk) disable iff (!rst_n)  
    (spi_state == TRANSFER && CPHA==0  
    && CS_low && `$rose(SCLK))  
    |-> ##1 (MOSI == `$past(MOSI,1));  
endproperty  
assert property (p_mosi_stable) else
```

```
$`error("MOSI changed on wrong edge");
```

This ensures that in mode 0, after a rising SCLK, MOSI stays constant until the next falling edge.

MISO Tri-state Check (Slaves): For a slave model, assert that when SS_n is high (slave deselected), its MISO output is Z or inactive:

```
systemverilog
```



```
assert property @(posedge clk) disable  
iff (!rst_n)  
    (CS_high && mode==SLAVE) |->  
    (MISO_z == 1);
```

Verifies the slave doesn't drive MISO when not selected.

FIFO Boundaries: Assertions can check FIFO

flags. E.g. `assert (TX_empty && start_transfer) == 0`; ensures we don't start transferring with an empty TX buffer.

Liveness/Safety: We also consider formal properties. A safety property: *"It is not possible for both MOSI and MISO to drive simultaneously to opposite values on the same net"*. A liveness property: *"Whenever SS_n goes low and data is presented, eventually a transfer done flag will be raised"*.

While SVA doesn't replace coverage, it catches corner-case bugs. Even simple assertions like `CS_n |-> spi_busy` (SS active implies busy flag) can catch mis-states. For each critical control signal, writing an assertion helps lock down the correct protocol.

Code examples above are indicative; actual implementation will depend on the design's internal signals and clocking.

11. Functional Coverage and Closure

To prove thorough testing, define a covergroup model for SPI. Coverpoints should include:

Mode Coverage: Cover CPOL and CPHA combinations (all four modes). Cross CPOL×CPHA to hit modes 0–3. Also cover MSB/LSB bit order if supported.

Bit-Width Coverage: Cover different word lengths (e.g. [8](#), [16](#), [32](#) bits) if variable.

Transfer Types: Cover single-byte, multi-byte

bursts, and continuous transfers. Cover the case of zero-length (e.g. just opcode, no data).

Slave-Select Coverage: For a multi-slave design, cover selecting each slave device and the case of no slave active.

Error Conditions: Cover mode fault occurrence, FIFO underflow/overflow events.

Edge and Setup Conditions: Cover SCLK rising vs falling, maybe even metastability injection (if possible in coverage).

A simple covergroup might look like:

```
systemverilog
```



```
covergroup cg_spi_transfer @(posedge  
clk);  
  coverpoint cpol { bins cp0 = {0}; bins  
cp1 = {1}; }
```

```
coverpoint cpha { bins ph0 = {0}; bins  
ph1 = {1}; }  
coverpoint bitrate { bins speeds[] =  
{8,16,32,64,128}; }  
coverpoint spi_len { bins len8 = {8};  
bins len16 = {16}; bins len_any = {  
[1:255] }; }  
cross cp_mode = (cpol, cpha);  
endgroup
```

Example: CPOL and CPHA coverpoints, with a cross to hit all 4 mode combinations.

Continue with crosses: e.g. length × direction (read/write), chip-select id × op code, or even pattern cover (specific data patterns). Aim for 100% coverage of all clauses in the protocol specification.

Closure Strategy: Plan to create tests to close

each coverpoint/bin. For example, if “spi_len” has bins up to 255, write tests for lengths 1,8,64,128,255. For error bins, create directed test for mode fault. Formal methods (like bounded model checking) can also complement coverage by proving unreachable states or covering some sequences exhaustively.

By combining constrained-random sequences with these covergroups, one can quantify progress. Don't forget cross-coverage: e.g. CPOL×CPHA×bit-order. All bins for “spi_len” × “transfer type” should be exercised.

12. Performance Analysis

Throughput and latency are key metrics for SPI interfaces:

Bitrate (Mbps): Determined by SCLK frequency and mode. In Mode 0/2, a bit is transferred on every rising edge; in Mode [1/3](#), on falling edge. A typical formula:
$$\text{Throughput (MB/s)} = \frac{f_{\text{SCLK}}}{8 \times 10^6} \times \frac{\text{bits per transfer}}{8}$$
 For example, a 50 MHz SCLK transfers 50 Mbit/s = 6.25 MByte/s of data (one byte per 8 clocks). Dual or quad modes effectively multiply this (2× or 4× data lines).

Latency: Time from when SS_n goes active to when first bit is available. In many flash reads, there is an inherent latency of *command + address* time plus dummy cycles. For example, in Fast Read, if SCLK=50 MHz (20 ns), sending

opcode+address (32 bits) takes ~640 ns, then 8 dummy clocks adds 160 ns, so the first data byte arrives ~800 ns after SS_n. Calculate total latency = (number of command/address bits + dummy) / SCLK.

Master/Slave Delay: If the SPI core uses an internal divider, there may be additional latency between CPU command and first SCLK. For instance, if the core is clocked at 100 MHz but the SPI divider divides by 2 to run at 50 MHz, each SCLK toggles every 40 ns.

Worked Example: Suppose an SPI master clocked at 100 MHz with an 8-bit shift. To send 100 bytes, SCLK = 25 MHz (divider = 4). The data transfer time is: 100 bytes × 8 bits/

byte = 800 SCLK cycles $\rightarrow 800/25\text{e}6 = 32$ μs . Add, say, 1 μs overhead for command + addressing. So total ~ 33 μs . Throughput ≈ 100 bytes / 33 $\mu\text{s} \approx 3$ MB/s.

Engineers should derive such formulas for their specific hardware (taking into account idle cycles between transactions if any). Also consider bus contention and multi-master arbitration if present (which add overhead).

13. Debugging Methodology

Waveform analysis is invaluable for SPI issues. Key signals to observe: SS_n, SCLK, MOSI, MISO, and any IRQ/status lines. A good debug approach:

Check Idle States: Verify that SS_n is high when idle, SCLK is at idle level (0 or 1 per CPOL), MOSI/MISO are idle (often HI-Z or pull-up).

Step-by-step Tracing: For each byte, examine clock edges. Make sure MOSI bits align with expected data. On which edge does the controller sample MOSI? Is it matching CPHA? Check if the sampled data matches the transmitted pattern.

Slave-Select Timing: Look at SS_n asserted before the first bit, and that it stays low for the entire transfer. If SS_n toggles early, it will abort.

Glitch Inspection: Pause or single-step simulation right before SS_n deasserts to see if the last bit was properly latched.

Throughput Checks: Count SCLK pulses to ensure it matches the configured bit length.

If fewer pulses occur than expected, you may have an internal “last bit” bug.

Status Monitoring: If available, watch the controller’s “busy” or “transfer-done” flags. Ensure they go active only during a transaction.

Use the built-in assertion diagnostics (from the SVA properties) to catch violations at runtime. Cross-probe the DUT state: e.g. if an assertion in the testbench says “state should be IDLE”, check the RTL’s state flip-flops to diagnose mis-encoding. In silicon bring-up, logic analyzers or on-chip trace (like ARM ETM) can help capture SPI lines. Overlay expected traffic (often by replaying a UVM scenario) to pinpoint mismatches.

14. Interview Q&A Examples

1. Q: What wires are needed for SPI?

A: Four: SCLK (clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), and SS_n (Slave Select, active low).

2. Q: Explain SPI Mode 0 and Mode 3 differences.

A: Both sample on the first edge, but in [Mode 0 \(CPOL=0\) SCLK idles low](#) (first edge is rising) whereas in Mode 3 (CPOL=1) SCLK idles high (first edge is falling).

3. Q: Can SPI communicate full-duplex?

A: Yes. On each clock, the master outputs a

bit on MOSI while the slave outputs a bit on MISO. After 8 clocks, both have exchanged a full byte.

4. Q: How do dual and quad SPI work?

A: After sending a command and address on the single MOSI line, the flash can output data on 2 or 4 lines (IO0–IO1 or IO0–IO3) simultaneously. Dual SPI (opcode 0x3B) gives 2× throughput; Quad SPI (opcode 0x6B) gives 4×. These require enabling the flash's QE bit.

5. Q: What is a mode-fault in SPI?

A: It's an error condition when another master drives the bus. If a master sees its SS_n pulled low unexpectedly (e.g., another controller activated), it flags MODE_FAULT.

6. Q: How to verify CPOL/CPHA in RTL?

A: Write SVA: e.g. in Mode 0 assert MOSI only changes on falling edges and is stable on rising edges. Similarly ensure sampling happens at the configured phase.

7. Q: How does a slave know when to output MISO?

A: It monitors SS_n. When SS_n goes low, the slave's FSM enters an active state and begins shifting out data from its shift register on MISO each clock, as master clocks SCLK.

8. Q: Describe how to handle unaligned byte transfers.

A: Most SPI cores automatically align to byte boundaries. If user needs sub-byte, the driver

Always test when transferring non-multiples of 8 to ensure no residual data.

9. Q: What checks would you add in UVM coverage?

A: Cover each CPOL/CPHA combination, MSB/LSB selection, byte lengths, and transfer types (single vs burst).

10. Q: Why are dummy cycles used in SPI flash reads?

A: To give the flash memory time to fetch data internally. Fast Read commands add N dummy clocks after the address so that the first data byte is available on the next clock.

15. Design and Verification Checklists

Design Checklist

Signal Definitions: SCLK, MOSI, MISO, SS_n correctly named and directioned.

Clocking: CPOL/CPHA modes implemented; clock divider supports required frequency range. SS_n gating stops SCLK when idle.

State Machine: Proper IDLE, TRANSFER, and COMPLETE states. SS_n only active during transfer.

Shift Registers: Data width and order (MSB/LSB) as specified. Double-buffered if needed (TX/RX buffers).

Multi-Slave: Sufficient number of SS lines or decoder. Unused slaves tristate MISO.

Interrupts/Flags: TX Empty, RX Full, Transfer Done, Mode Fault flags implemented.

Error Handling: Mode-fault, overrun, underrun conditions handled (e.g. state reset or stall).

CRC/Parity (if any): Some SPI blocks support CRC on transfers (verify if needed).

Clock Domain Crossing: SS_n and asynchronous inputs synchronized. No combinatorial loops.

Verification Checklist

Environment Setup: Virtual interfaces for SPI signals; both master and slave agents.

Basic Transfers: Read/write single byte, multi-byte bursts; all CPOL/CPHA modes.

Edge Cases: Zero-length transfer, max-length transfer, misaligned (if allowed).

Master/Slave Roles: Test both master operation and slave response (loopback or reference model).

Multi-Slave and Daisy-Chain: If multi-slave, select each; if daisy, chain sequence.

Timing Corners: Clock divider = 0 (fastest), slowest clock; SS_n timing (hold time violations).

Fault Injection: Toggle SS_n mid-transfer, cause FIFO underflow/overflow, mode fault.

Formal Checks: Run assertions and formal properties.

Functional Coverage Checklist

Modes: All CPOL/CPHA combinations (cross coverage).

Bit Orders: MSB-first vs LSB-first.

Transfer Lengths: Typical (8b) and others

(16b, 32b, random lengths).

Slave-Select: Each slave device (1..N), and no-selection case.

Command Types: Read, write, fast read, dual/quad, etc. (Where applicable).

Error/Exception: Mode fault occurred or not; error condition happened.

Signoff Criteria

All assertions pass with no unexpected failures in regression.

100% coverage of the covergroups above.

All directed tests' expected data match observed data.

Timing diagrams match hand-wave calculations and datasheet limits.

No violation of setup/hold in static timing analysis (consider worst-case PVT).

Code reviews confirm CPOL/CPHA handling and data path integrity.

Sources: Authoritative protocol descriptions and device datasheets were used throughout. These ensure that SPI modes, signal behavior, and flash command sequences are correctly reflected in the tutorial content. All figures and tables are derived from or consistent with these references.